

5

AGENTIC PATTERNS

Every AI Engineer Must Know

The design blueprints behind every
intelligent AI system.

 Think →  Plan →  Act



AI Coach John

Follow



PROITBRIDGE
DRIVE THE BETTER FUTURE

 Repost

What Are **Agentic Patterns**?

Without patterns, you're just chaining prompts and hoping for the best. **Agentic patterns are proven architectural blueprints for how AI agents think, plan, act, and collaborate.**

THE BASICS

What: Reusable architectural designs for agent behavior

Why: Without them: brittle pipelines, no error recovery, no coordination

When: Any time your task needs planning, tool use, multi-step reasoning, or multiple AI actors

How: Pick the right pattern for the right problem. This carousel covers all 5.

REAL SCENARIO

You build a customer support agent. It answers FAQs fine. Then a user asks: *"Check my order, raise a refund, and send me a confirmation email."*

A single prompt chain collapses. You need an agent that plans, uses tools, and executes steps in sequence.

THE 5 PATTERNS

- > **ReAct:** Reason + Act in a loop
- > **Plan & Execute:** Plan first, then act
- > **Reflection:** Self-critique and improve
- > **Tool Use:** Extend via external tools
- > **Multi-Agent:** Specialized collaboration



ReAct (Reason + Act)

CONCEPT

What: Agent alternates between Reasoning and Acting in a continuous loop.

Why: Combines chain-of-thought with real-world tool use. Grounded reasoning, not hallucination.

When: Tasks needing iterative lookup, calculation, or live data fetching.

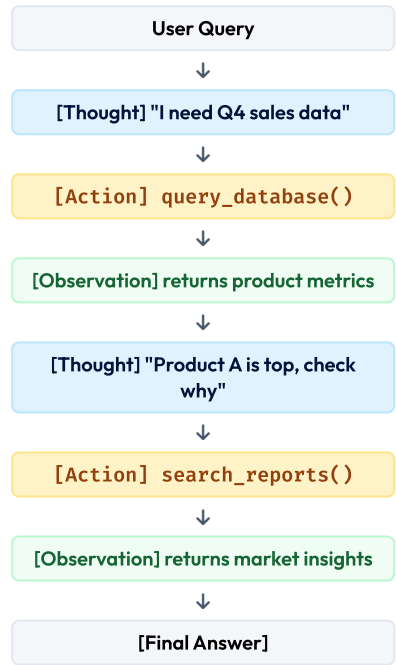
How: Each loop = Thought → Action → Observation → repeat until final answer.

SCENARIO

A data analyst agent answering: *"What was our best performing product last quarter and why?"*

Can't answer in one shot. Needs to reason, query, observe, reason again, repeat. ReAct handles this naturally: think, act, observe, loop.

ARCHITECTURE



FRAMEWORKS

 LangChain ReAct

 LlamaIndex ReAct



Plan & Execute

CONCEPT

What: Agent generates a complete step-by-step plan first, then executes each step in order.

Why: Separates thinking from doing avoids reactive drift on complex tasks.

When: Long-horizon tasks with clear subtasks: reports, research, trip planning, code generation.

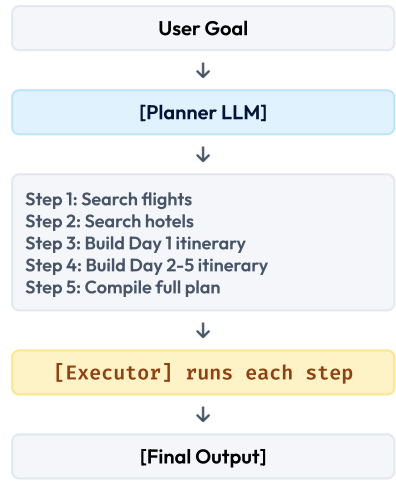
How: Planner LLM creates task list → Executor LLM handles each task one by one.

SCENARIO

An AI travel planner: *"Plan a 5-day Japan trip with flights, hotels, and itinerary"*

ReAct would drift reactively between flights, hotels, and day plans. Plan & Execute builds the full plan first, then executes cleanly. Far more reliable for complex deliverables.

ARCHITECTURE



FRAMEWORKS

 LangChain Plan-and-Execute

 LangGraph Planner Node



Reflection (Self-Critique)

CONCEPT

What: Agent generates output → critiques it → revises → repeats until satisfied.

Why: Self-correction without human feedback. Catches errors, improves quality autonomously.

When: Writing, code generation, reasoning tasks where first output is rarely best.

How: Generator produces draft → Critic scores and lists weaknesses → Generator revises → loop.

SCENARIO

An AI agent writing a technical blog post.

First draft is generic, shallow, misses key points. Without reflection that draft ships. With reflection, agent critiques its own output, identifies gaps, rewrites, iterates until quality threshold met. Output goes from 6/10 to 9/10 without human intervention.

ARCHITECTURE



FRAMEWORKS

 **LangGraph Reflexion Loop**

 **AutoGen**

 **Reflexion Paper Impl.**



Tool Use

CONCEPT

What: Agent equipped with external tools it can call APIs, DBs, calculators, browsers, code runners.

Why: LLMs alone are frozen in weights. Tools give them real-world reach and live data.

When: Any task needing live data, computation, file access, or external system interaction.

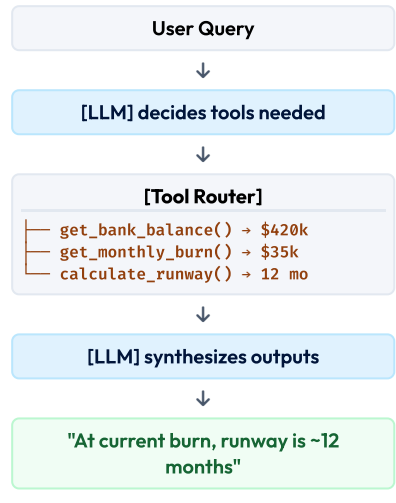
How: Define tools with clear names and descriptions → LLM decides when and which tool to call.

SCENARIO

An AI CFO assistant: *"What's our cash runway given last month's burn rate?"*

LLM alone can't answer no access to live financials. With tools it fetches balance, fetches burn rate, calculates runway, returns answer in seconds. Tools make agents actually useful.

ARCHITECTURE



FRAMEWORKS

- LangChain Tools
- OpenAI Functions
- LlamaIndex Tools
- LangGraph ToolNode



Multi-Agent

CONCEPT

What: Multiple specialized agents, each with a defined role, coordinated by an orchestrator.

Why: Division of labor. Specialized agents outperform one generalist on complex tasks.

When: Tasks with distinct subtasks that benefit from specialization, parallelism, or checks and balances.

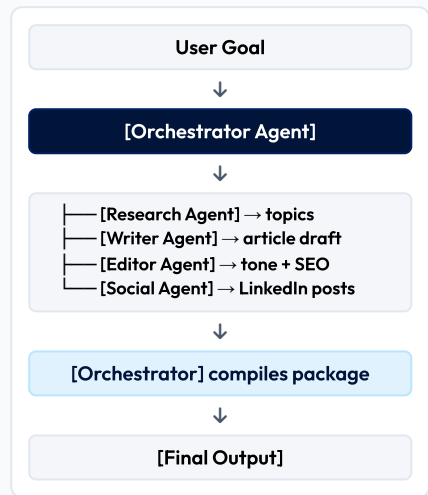
How: Orchestrator receives goal → assigns subtasks to specialists → aggregates outputs.

SCENARIO

Automating a full content marketing pipeline:

One agent researches trending topics. One writes the draft. One edits for tone and SEO. One generates social posts. Orchestrator coordinates all of them end to end. No single agent does this reliably specialization wins every time.

ARCHITECTURE



FRAMEWORKS

 CrewAI

 LangGraph Multi-Agent

 AutoGen

 OpenAI Swarm



Patterns Cheatsheet

PATTERN	BEST FOR	CORE LOOP
 ReAct	Live data, iterative lookup	Think → Act → Observe
 Plan & Execute	Long multi-step tasks	Plan → Execute in order
 Reflection	Writing, code, quality output	Generate → Critique → Revise
 Tool Use	Real-world data and computation	Query → Tool Call → Synthesize
 Multi-Agent	Complex specialized tasks	Orchestrate → Delegate → Aggregate

Master these patterns.

They are the foundation of all advanced AI systems being built today.



AI Coach John

Follow



PROITBRIDGE
DRIVEN BY BETTER FUTURE

↻ Repost